

PROJECT REPORT

CIFAKE AI DETECTOR

AI-Generated Image Detection Using Deep Learning and Explainable AI

Final Accuracy: 96.24% | ROC-AUC: 0.9946

 **Live Demo**

Frontend: <https://cifake-ai-detector.vercel.app/>

Backend API: <https://cifake-ai-detector.onrender.com/>

API Health Check: <https://cifake-ai-detector.onrender.com/health>

Project Done By:
Singupurapu Srihari

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid development of Artificial Intelligence has made it possible to generate highly realistic images using technologies such as Generative Adversarial Networks (GANs), diffusion models, and other generative AI techniques. AI-generated images are increasingly being used in areas such as digital art, content creation, education, advertising, entertainment, and social media.

Although these technologies provide many useful applications, they also create challenges in distinguishing between real images and artificially generated images. AI-generated content can sometimes appear visually similar to authentic photographs, making manual identification difficult.

The **CIFAKE AI Detector** is a deep-learning-based image classification system designed to distinguish between real and AI-generated images. The system uses a custom Convolutional Neural Network (CNN) trained on the CIFAKE dataset. The model classifies an input image into two categories:

- REAL – an image belonging to the real-image class.
- FAKE – an image generated synthetically by an AI model.

The project also incorporates Grad-CAM (Gradient-weighted Class Activation Mapping) to provide visual explanations for the model's predictions. A FastAPI backend handles model inference and image processing, while a React-based frontend provides an interactive interface for users.

The final system achieved **96.24% accuracy** and a **0.9946 ROC-AUC** on the official CIFAKE test dataset.

1.2 Problem Statement

The increasing availability of AI image-generation technologies has made it difficult to determine whether an image is authentic or synthetically generated.

Traditional manual inspection is unreliable because AI-generated images can contain realistic textures, lighting, objects, and other visual characteristics.

Therefore, there is a need for an automated system capable of analyzing images and classifying them as real or AI-generated while also providing some explanation of the model's decision.

1.3 Objectives

The main objectives of the project are:

1. To develop a deep-learning-based system for detecting AI-generated images.
2. To train a CNN model using the CIFAKE dataset.
3. To evaluate the model using accuracy, precision, recall, F1-score, and ROC-AUC.
4. To implement Grad-CAM for visual model explainability.
5. To develop a FastAPI backend for model inference.

6. To develop a React frontend for user interaction.
7. To integrate the machine-learning model with a complete web application.
8. To demonstrate an end-to-end AI image classification pipeline.

CHAPTER 2: SYSTEM OVERVIEW

2.1 Overview

The CIFAKE AI Detector is an end-to-end machine-learning application consisting of four major components:

9. Dataset and preprocessing
10. Deep-learning model
11. FastAPI backend
12. React frontend

The system receives an image from the user, preprocesses it according to the model requirements, passes it through the trained CNN, generates a REAL/FAKE prediction, and produces a Grad-CAM visualization explaining the prediction.

2.2 Existing System

Existing approaches for identifying AI-generated images include:

- Manual visual inspection
- Image metadata analysis
- Digital watermarking
- Traditional image-processing techniques
- Machine-learning-based classifiers
- Deep-learning-based detectors

Manual inspection becomes increasingly difficult as generative models produce more realistic images.

Metadata-based approaches can also be unreliable because metadata may be removed or modified when images are shared online.

2.3 Proposed System

The proposed system uses a custom CNN trained specifically for binary classification of real and AI-generated images.

The proposed pipeline is:

```
Input Image → Preprocessing → CNN Model → Prediction → Grad-CAM → Result
```

The system provides:

- REAL/FAKE classification
- Prediction confidence
- Fake probability

- Grad-CAM visualization
- REST API
- Web-based interface

2.4 System Workflow

The overall workflow is:

13. User uploads an image.
14. The frontend sends the image to the FastAPI backend.
15. The backend reads and preprocesses the image.
16. The image is resized to 32×32 pixels.
17. The processed image is passed to the trained CNN.
18. The model generates a probability using the sigmoid output.
19. The probability is converted into a REAL or FAKE classification.
20. Grad-CAM generates a visual explanation.
21. The backend returns the prediction and Grad-CAM result.
22. The React frontend displays the result to the user.

CHAPTER 3: LITERATURE SURVEY

3.1 AI-Generated Image Detection

The rapid development of generative AI has resulted in research focused on automatically identifying synthetic images. Researchers have explored traditional machine-learning techniques, CNN-based approaches, frequency-domain analysis, image artifacts, and deep-learning-based classification.

Deep-learning models are particularly useful because they can automatically learn visual features from training data rather than requiring all detection features to be manually designed.

3.2 Convolutional Neural Networks for Image Classification

Convolutional Neural Networks are widely used in computer vision applications because they can learn hierarchical visual representations.

Initial layers can learn low-level features such as:

- Edges
- Corners
- Textures

Deeper layers can learn more complex patterns and combinations of features.

For this project, a custom CNN was developed instead of directly using a large pretrained architecture.

3.3 CIFAKE Dataset

The CIFAKE dataset provides a benchmark for distinguishing real images from synthetic images.

The dataset contains:

- 60,000 real images
- 60,000 AI-generated images
- 120,000 images in total

The real images are derived from CIFAR-10, while the synthetic images were generated using Stable Diffusion v1.4.

The original images have a resolution of 32×32 pixels, making the dataset suitable for experimentation with lightweight CNN architectures.

3.4 Explainable Artificial Intelligence

Deep-learning models are often considered difficult to interpret because their internal decision-making process is not directly visible.

Explainable AI techniques help provide insight into model decisions.

Grad-CAM is one such technique. It produces a heatmap showing image regions that contributed to a CNN's prediction.

In this project, Grad-CAM was integrated to improve the interpretability of the classification system.

CHAPTER 4: TECHNOLOGIES USED

4.1 Programming Languages

Python

Python was used for:

- Dataset processing
- Model development
- Model training
- Evaluation
- Grad-CAM
- Backend development

JavaScript

JavaScript was used for the React frontend.

4.2 Machine Learning Technologies

TensorFlow

TensorFlow was used as the primary deep-learning framework.

Keras

Keras was used to define and train the CNN architecture.

Scikit-learn

Scikit-learn was used for:

- Classification metrics
- Confusion matrix
- ROC-AUC
- Evaluation utilities

NumPy

NumPy was used for numerical operations and image-array processing.

OpenCV / Pillow

Image-processing libraries were used for image loading, resizing, and preprocessing.

4.3 Backend Technologies

FastAPI

FastAPI was used to create the REST API responsible for:

- Receiving uploaded images
- Running model inference
- Generating Grad-CAM
- Returning prediction results

Uvicorn

Uvicorn was used as the ASGI server for running the FastAPI application.

4.4 Frontend Technologies

React

React was used to build the user interface.

Vite

Vite was used as the frontend development and build tool.

4.5 Development Environment

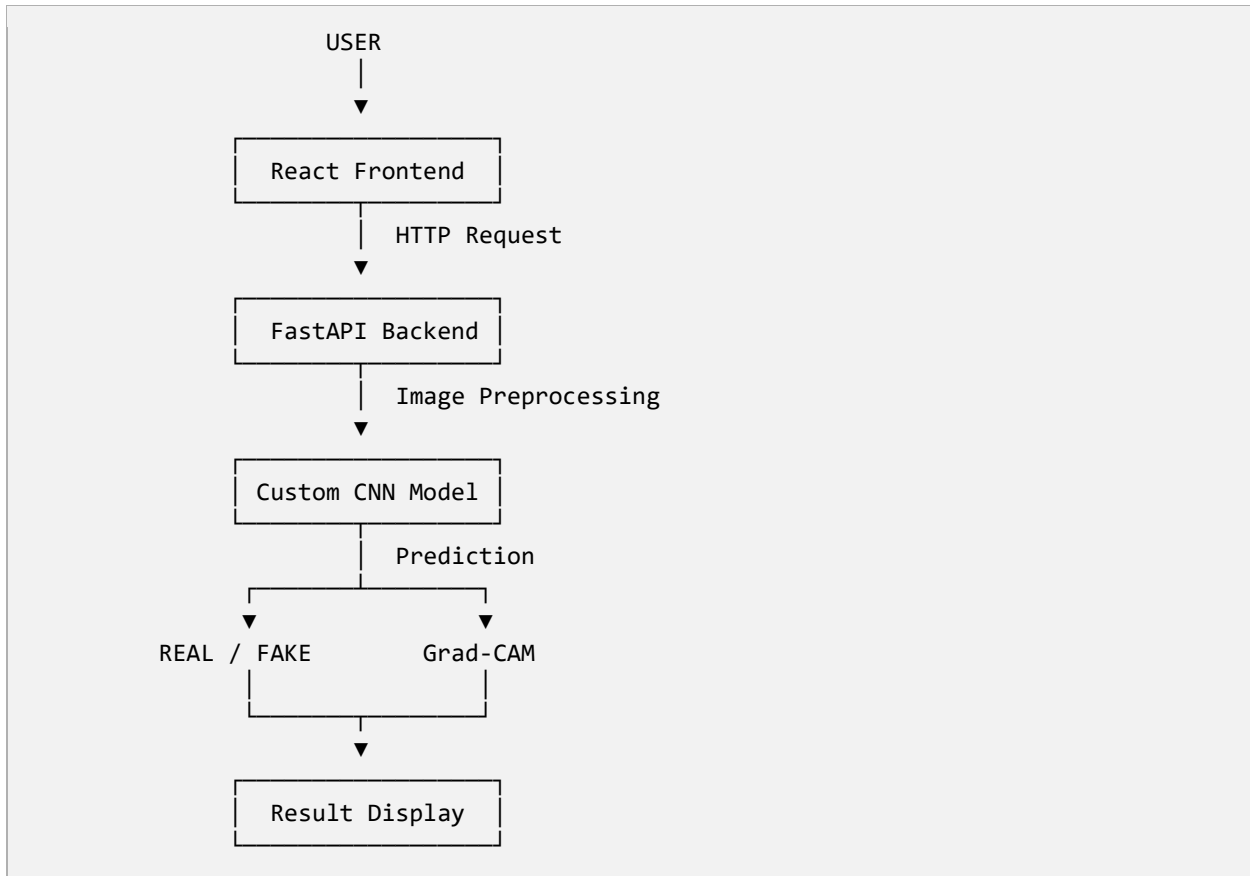
The model training environment used:

- Python 3.11
- TensorFlow 2.21
- WSL2 Ubuntu
- NVIDIA CUDA-enabled TensorFlow
- NVIDIA RTX 4050 Laptop GPU with 6 GB VRAM
- Visual Studio Code

CHAPTER 5: ARCHITECTURE AND DESIGN

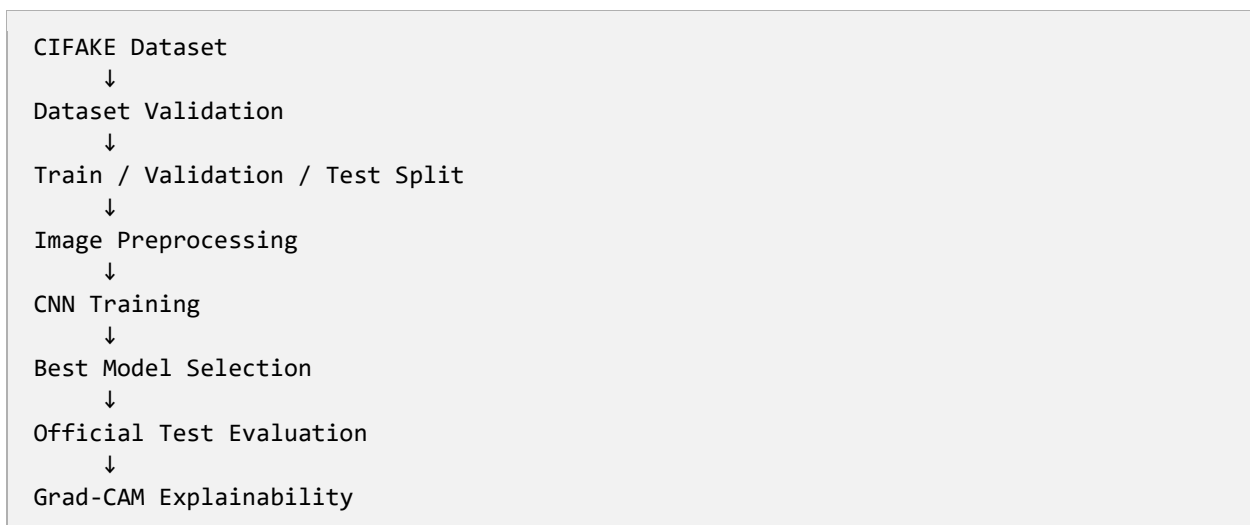
5.1 System Architecture

The architecture consists of the following layers:



5.2 Machine Learning Pipeline

The machine-learning pipeline is:



5.3 Dataset Split

The dataset was divided as follows:

Dataset	Total Images	REAL	FAKE
Training	90,000	45,000	45,000
Validation	10,000	5,000	5,000
Testing	20,000	10,000	10,000
Total	120,000	60,000	60,000

Table 5.1 — Dataset split across training, validation, and testing sets

The official test set was kept separate and untouched during training.

5.4 CNN Architecture

The implemented CNN contains three major convolutional blocks.

Block 1

- Conv2D – 32 filters
- Batch Normalization
- Conv2D – 32 filters
- Max Pooling
- Dropout

Block 2

- Conv2D – 64 filters
- Batch Normalization
- Conv2D – 64 filters
- Max Pooling
- Dropout

Block 3

- Conv2D – 128 filters
- Batch Normalization
- Conv2D – 128 filters
- Max Pooling
- Dropout

Classification Head

- Global Average Pooling
- Dense layer – 128 neurons
- Dropout
- Sigmoid output

The model contains approximately **304,545 trainable parameters**.

5.5 Prediction Logic

The final sigmoid layer produces a value between 0 and 1.

The system interprets the value as the probability of the image being FAKE.

Probability $\geq 0.5 \rightarrow$ FAKE

Probability $< 0.5 \rightarrow$ REAL

The confidence is calculated according to the predicted class.

CHAPTER 6: IMPLEMENTATION

6.1 Dataset Preparation

The CIFAKE dataset was organized into REAL and FAKE categories.

The dataset was validated before training to ensure:

- Correct class distribution
- No missing files
- Correct train/validation/test separation
- No overlap between training and validation image paths

6.2 Image Preprocessing

Each image is:

23. Loaded using PIL.
24. Converted to RGB.
25. Resized to 32×32 pixels.
26. Converted into a NumPy array.
27. Passed to the TensorFlow model.

The preprocessing ensures that all images match the input shape expected by the CNN.

6.3 Model Training

The CNN was compiled using:

- Optimizer: Adam
- Learning rate: 0.001
- Loss: Binary Cross-Entropy
- Metrics: Accuracy and ROC-AUC

Training included:

- Model checkpointing
- Early stopping
- Learning-rate reduction
- Validation monitoring

The best-performing model was saved as:

```
cifake_custom_cnn_best.keras
```

6.4 Model Evaluation

After training, the best model was restored and evaluated against the official 20,000-image test set.

The evaluation included:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix
- ROC curve
- Prediction distribution

6.5 Grad-CAM Implementation

Grad-CAM was implemented using TensorFlow's GradientTape.

The system obtains:

28. Feature maps from the final convolutional layer.
29. Gradients of the prediction.
30. Average gradient values as importance weights.
31. Weighted feature maps.
32. A normalized heatmap.

The heatmap is then combined with the original image to produce a visual explanation.

6.6 FastAPI Backend

The FastAPI backend provides the following endpoints:

GET /

Provides basic API information.

GET /health

Checks whether the backend is running correctly.

POST /predict

Accepts an image and returns:

- Filename
- Prediction
- Confidence
- Fake probability

- Grad-CAM URL

Example response:

```
{
  "filename": "image.png",
  "prediction": "FAKE",
  "confidence": 0.9621,
  "probability_fake": 0.9621,
  "gradcam_url": "/static/gradcam/example_gradcam.jpg"
}
```

6.7 React Frontend

The React frontend provides:

- Image upload interface
- Loading state
- Prediction result
- Confidence information
- Grad-CAM visualization
- Error handling

The frontend communicates with the FastAPI backend using HTTP requests.

6.8 GPU Configuration

TensorFlow was configured to use the NVIDIA RTX 4050 GPU through WSL2.

GPU availability was verified using:

```
tf.config.list_physical_devices("GPU")
```

The GPU was successfully detected as:

```
PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')
```

This allowed the CNN training process to use GPU acceleration.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Model Performance

The trained CNN was evaluated on the official 20,000-image test set. The obtained results were:

Metric	Result
Accuracy	96.24%
Precision	97.65%
Recall	94.75%
F1-Score	96.18%
ROC-AUC	99.46%

Table 7.1 — Model performance on the official test set

7.2 Classification Report

	precision	recall	f1-score	support
REAL	0.9490	0.9772	0.9629	10000
FAKE	0.9765	0.9475	0.9618	10000
accuracy			0.9624	20000
macro avg	0.9628	0.9624	0.9623	20000
weighted avg	0.9628	0.9624	0.9623	20000

7.3 Confusion Matrix

The confusion matrix was:

	Predicted REAL	Predicted FAKE
Actual REAL	9,772	228
Actual FAKE	525	9,475

Table 7.2 — Confusion matrix on the official 20,000-image test set

The model correctly classified:

- 9,772 REAL images
- 9,475 FAKE images

The model incorrectly classified:

- 228 REAL images as FAKE
- 525 FAKE images as REAL

7.4 ROC-AUC

The model achieved a ROC-AUC of **0.9946**.

This indicates strong separation between the REAL and FAKE classes on the CIFAKE test distribution.

7.5 Grad-CAM Results

Grad-CAM was successfully integrated into the application.

The visualization highlights image regions that contributed to the CNN's prediction.

This improves interpretability because users can see not only the classification result but also a visual representation of where the model focused.

7.6 End-to-End System Result

The complete system was successfully integrated:

```
Image Upload → React Frontend → FastAPI → TensorFlow CNN →  
Prediction → Grad-CAM → Result + Explanation → React UI
```

The local end-to-end system was successfully tested.

7.7 Generalization Observation

During additional testing, an external AI-generated image from a distribution different from CIFAKE was incorrectly classified as REAL.

This demonstrated an important limitation of the current system: **high performance on the CIFAKE test set does not guarantee universal AI-image detection.**

The result indicates that the model can be affected by distribution shift and unseen image-generation methods.

Therefore, the current system should be considered a CIFAKE-trained AI-image detection prototype rather than a universal AI detector.

CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

8.1 Challenges Faced

1. TensorFlow GPU Configuration

Initially, TensorFlow was not utilizing the GPU effectively. Setting up TensorFlow with CUDA support under WSL2 required configuring the required NVIDIA libraries and environment variables.

The final setup successfully detected the RTX 4050 GPU.

2. Large Dataset Handling

The CIFAKE dataset contains 120,000 images. Managing the dataset and maintaining separate training, validation, and test sets required careful organization.

3. Model Training

Training a CNN on a large image dataset required GPU acceleration and appropriate training strategies.

4. Model Explainability

Generating meaningful explanations required understanding the relationship between CNN feature maps and gradients.

Grad-CAM was implemented to solve this problem.

5. Frontend-Backend Integration

Connecting React with FastAPI introduced issues related to API communication and CORS configuration.

The issue was resolved by configuring FastAPI's CORS middleware.

6. Generalization

Testing with an external AI-generated image revealed that the model does not automatically generalize to every type of AI-generated image.

This helped identify an important real-world limitation of the current model.

8.2 Learning Outcomes

Through this project, the following concepts were learned and implemented:

Machine Learning

- Binary classification
- Dataset preparation
- Training and validation
- Model evaluation
- Overfitting prevention
- Performance metrics

Deep Learning

- CNN architecture
- Convolutional layers
- Pooling
- Batch normalization
- Dropout
- Global average pooling
- Sigmoid classification

Explainable AI

- Grad-CAM
- Gradient-based visualization
- Model interpretability

Backend Development

- FastAPI
- REST APIs
- File uploads
- JSON responses
- CORS
- Model serving

Frontend Development

- React
- Component-based architecture
- API integration
- State management
- Dynamic result rendering

GPU Computing

- WSL2
- TensorFlow GPU
- CUDA-enabled deep learning
- NVIDIA GPU configuration

Software Engineering

- Project organization
- Environment variables
- Requirements management
- Git/GitHub preparation
- Documentation
- End-to-end system integration

CHAPTER 9: FUTURE ENHANCEMENTS

The current CIFAKE detector can be further improved in several ways.

9.1 Larger and More Diverse Dataset

The current model is trained primarily on the CIFAKE distribution.

A future version can include images generated by multiple AI systems such as:

- Stable Diffusion
- DALL-E
- Midjourney
- Flux
- Other modern image-generation models

Real images from multiple sources can also be included.

9.2 Higher Resolution Images

The current CIFAKE images have a resolution of 32×32 pixels.

Future versions can use higher-resolution images such as:

- 128 × 128
- 224 × 224
- 256 × 256

Higher resolution can preserve more visual information.

9.3 Transfer Learning

Future versions can compare the custom CNN with pretrained architectures such as:

- ResNet
- EfficientNet
- MobileNet
- Vision Transformers

This could provide stronger feature extraction for more diverse datasets.

9.4 Improved Generalization

The model should be evaluated using completely unseen AI generators.

This would provide a better measurement of real-world generalization.

9.5 Ensemble Models

Multiple models could be combined to improve robustness. For example:

```
CNN + EfficientNet + Vision Transformer → Ensemble Prediction
```

9.6 Production Deployment

The system can be deployed as a public web application with:

- Cloud-hosted backend
- Cloud-hosted frontend
- HTTPS
- Authentication
- Request limits
- Logging
- Monitoring

9.7 Additional Explainability

Future versions can include additional explainability methods alongside Grad-CAM to provide more detailed analysis of model decisions.

CHAPTER 10: CONCLUSION

The **CIFAKE AI Detector** project demonstrates the development of an end-to-end deep-learning system for detecting AI-generated images.

A custom Convolutional Neural Network was developed and trained using the CIFAKE dataset containing 120,000 real and AI-generated images. The dataset was carefully divided into training, validation, and official test sets.

The trained model achieved **96.24% accuracy, 97.65% precision, 94.75% recall, 96.18% F1-score, and 0.9946 ROC-AUC** on the official 20,000-image test set.

To improve model interpretability, **Grad-CAM** was integrated to visualize the image regions contributing to predictions.

The machine-learning model was then integrated into a complete software application using **FastAPI** as the backend and **React** as the frontend. This resulted in an end-to-end system where users can upload an image, receive a REAL/FAKE prediction, view the confidence, and inspect a Grad-CAM visualization.

The project also demonstrated an important machine-learning limitation. Testing with an external AI-generated image showed that strong performance on a particular benchmark dataset does not guarantee generalization to unseen image-generation methods. This highlights the importance of diverse datasets and cross-generator evaluation when developing real-world AI-content detection systems.

Overall, the project provided practical experience in deep learning, computer vision, explainable AI, GPU computing, backend API development, frontend integration, model evaluation, and machine-learning system deployment. It also provides a foundation for future improvements involving larger datasets, higher-resolution images, multiple AI generators, transfer learning, and more robust real-world evaluation.